NLP 2025 Lab 2 Report: Group 35

RSAA (rowaid) Asaba
KM (Kordian) Klimas
Tikhon Magafurov

April 22, 2025

1 Exercise 1: Clean function

We implemented a text-cleaning function to standardize and preprocess the dataset. Specifically, the following steps were performed:

- Removing commas from numerical values (e.g., `15,000` becomes `15000`).
- Normalizing multiple whitespace characters to single spaces.
- Separating punctuation from words by adding spaces around punctuation.
- Converting all characters to lowercase for consistency.

Applying this cleaning process resulted in the following structured dataset:

```
DatasetDict({
    train: Dataset({
        features: ['set', 'clean_sentence', 'clean_compressed'],
        num_rows: 144000
    }),
    test: Dataset({
        features: ['set', 'clean_sentence', 'clean_compressed'],
        num_rows: 36000
    })
})
```

The dataset now includes clearly cleaned sentences in the fields `clean`

2 Exercise 2: Tokenize function

We tokenized the cleaned text using NLTK's `word_tokenize` function, chosen for its effectiveness in separating punctuation and special characters from words, which is crucial for accurate vocabulary building and embedding creation.

3 Exercise 3: Extracting Vocabulary Counts

To build our vocabulary, we counted the frequency of each token across the entire training dataset using Python's `Counter` class. This process provided clear insights into word usage and distribution, which informed decisions regarding vocabulary size and representation strategies.

The total number of unique words extracted was **119,369**. However, to reduce computational cost and focus on the most informative tokens, we limited the vocabulary size to the **10,000 most common words**. These were sorted in descending order based on frequency, allowing us to create a consistent and efficient Bag-of-Words (BoW) representation.

Below are examples of the most common tokens and their frequencies:

```
[('the', 183148), ('.', 177622), (',', 146033), ('to', 130603), ('a', 116944),
 ('in', 113133), ('of', 100519), ('and', 68453), ('on', 56441), ('for', 51910)]
```

The top 20 tokens and their assigned IDs (based on frequency ranking) are:

```
00: the   → 0
01: .     → 1
02: ,     → 2
03: to    → 3
04: a     → 4
05: in    → 5
06: of    → 6
07: and   → 7
08: on    → 8
09: for   → 9
10: has   → 10
11: with  → 11
12: 's    → 12
13: is    → 13
14: at    → 14
15: that  → 15
```

```
16: from  → 16
17: ``    → 17
18: by    → 18
19: as    → 19
20: new   → 20
```

These token-to-ID mappings were then used to construct BoW vectors for each sentence, ensuring that word representations are consistent and interpretable across all downstream tasks.

4 Exercise 4: Bag of Words

We implemented the Bag-of-Words (BoW) model to convert tokenized sentences into fixed-length numerical vectors. Each vector has a dimensionality equal to the vocabulary size, which in our case is **10,000**. This means every sentence is represented by a dense vector of length 10,000, where:

- A value of **1** at index i indicates the presence of the i-th most frequent word in the sentence.
- A value of **0** at index i indicates the word is not present.

This representation is simple yet effective for text similarity and retrieval tasks, as it captures word occurrence patterns in a binary or frequency-based format.

Example:

```
Tokenized sentence:
['maicon', 'has', 'completed', 'a', 'move', 'from', 'inter', 'milan',
 'to', 'manchester', 'city', ',', 'the', 'premier', 'league', 'club',
 'have', 'confirmed', '.']

Bag of words:
[1 1 1 ... 0 0 0]

Shape of bag of words:
(10000,)

Non-zero elements in bag of words:
[   0    1    2    3    4   10   16   33   70  180  283  424  425
  559  759 1154 1729 3695]
```

These non-zero indices correspond to the positions of the words in the vocabulary that appear in the sentence. This binary-style representation allows for efficient and scalable similarity computations.

5 Exercise 5: Cosine Similarity between Two Vectors

We implemented a function to compute the cosine similarity between two sentence vectors. Cosine similarity is a widely used metric in information retrieval and NLP for measuring the semantic similarity between two vectors regardless of their magnitude. The formula used is:

$$CosineSimilarity(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Where:

- $A \cdot B$ is the dot product of vectors A and B,
- $\|A\|$ and $\|B\|$ are the Euclidean norms (magnitudes) of A and B respectively.

To test the function, we applied it to a set of example sentences with the query: *"fox and deer"*. The cosine similarity results were:

```
Query: fox and deer

Cosine Similarity: 0.1741 - Sentence: The quick brown fox jumps over the lazy dog.
Cosine Similarity: 0.0000 - Sentence: Some interesting document containin sentences.
Cosine Similarity: 0.2981 - Sentence: The quick brown fox jumps over the lazy cat and some othe
Cosine Similarity: 1.0000 - Sentence: Fox and deer
Cosine Similarity: 1.0000 - Sentence: Fox and deer Fox and deer Fox and deer
Cosine Similarity: 1.0000 - Sentence: deer and Fox
Cosine Similarity: 0.6547 - Sentence: Fox and deer are not friends.
Cosine Similarity: 0.3522 - Sentence: Fox and deer are not friends. But this document is a lot
```

As expected, the exact or closely matching phrases received the highest similarity scores (1.0000), while unrelated sentences (like the second one) scored near zero.

We then mapped this cosine similarity function over our full dataset, enriching it with vector representations. The resulting dataset now includes BoW vectors and is structured as follows:

```
Dataset({
    features: ['set', 'clean_sentence', 'clean_compressed',
               'sentence_tokens', 'compressed_tokens',
               'sentence_bow', 'compressed_bow'],
})
```

This prepares the dataset for further similarity-based search and retrieval tasks.

6 Exercise 6: Cosine Similarity Between a Vector and an Array of Vectors

To calculate the similarity between a query and all other sentence vectors in the dataset, we extended our cosine similarity computation from a single pair of vectors to a full array of comparisons. This is critical for large-scale retrieval tasks, where we need to compare a query against thousands of sentences efficiently.

We implemented a function that computes the cosine similarity between one query vector and an array of sentence vectors using vectorized NumPy operations. Specifically, we avoided using loops by applying matrix multiplication and broadcasting:

$$cosine_similarity(q, M) = \frac{M \cdot q}{\|M\| \cdot \|q\|}$$

Where:

- q is the query vector of shape $(D,)$
- M is the matrix of sentence vectors of shape (N, D)
- $M \cdot q$ represents a dot product between the query and each sentence vector
- Norms are calculated and broadcasted to match dimensions

This efficient computation enables fast ranking of documents based on similarity.

Retrieval Example

We tested this function using the query `" and "` ¨. The resulting similarity scores for the first 30 sentences were:

```
[0.39223227 0.         0.         0.         0.20412415 0.12909944
 0.23570226 0.         0.         0.         0.         0.
```

```
0.18898224 0.         0.         0.25819889 0.         0.2773501
0.         0.         0.         0.34815531 0.27216553 0.
0.32025631 0.23570226 0.         0.21320072 0.         0.        ]
```

Among the full dataset, the sentence with the highest cosine similarity to the query was found at **index 29837**, with a cosine similarity score of **0.6882**. The sentence is:

> *"CW INDUSTRIES High density rectangular connector with enhanced retention and solderability, meets the requirements of MIL-DTL-28804 and NAVSEA 3164341 and 3164342 specifications."*

This demonstrates that the retrieval function is effective in identifying relevant matches based on BoW vector proximity.

7 Exercise 7: Analyzing and Improving BOW Search Results

We conducted an in-depth analysis of our Bag-of-Words (BoW) retrieval results and discovered both strengths and limitations. Below, we summarize our findings based on different query types, dataset characteristics, and linguistic phenomena.

Does the Search Perform Well? When Does It Fail?

Expected (Good) Results

1. **Query: "Fox news channel"**
   The query produced relevant results because the phrase "Fox news" is a specific and frequently occurring entity in the dataset. The presence of rare words like "channel" further improved the accuracy.

2. **Query: "prisoner in prison"**
   This query performed well since it included a meaningful combination of co-occurring words. The phrase effectively narrowed down the search space compared to using "prisoner" or "prison" alone.

3. **Query: "terrorist attack in London"**
   The use of relatively specific and contextually strong words like "terrorist" and "London" led to accurate and relevant results, highlighting the strength of BoW when rare terms are included.

Unexpected (Poor) Results

1. **Query: "fox"**
   Although intended to refer to the animal, most retrieved results involved "Fox News," demonstrating BoW's failure to disambiguate words with multiple meanings (polysemy).

2. **Query: "and"**
   As a highly frequent stopword, this resulted in extremely poor search results. Common conjunctions skew similarity scores due to their prevalence, even though they offer little semantic value.

3. **Query: "DRUNK neighbour attacked a woman"**
   The top result retrieved was: "A DRUNK woman attacked a neighbour with a windscreen wiper she had snapped off his car." Despite having all words, the word order is reversed, distorting the meaning. This highlights BoW's inability to capture word order or sentence structure.

Analysis of Failure Cases

- **Word ambiguity**: Words like "fox" or "bank" can have multiple meanings. BoW lacks contextual awareness, so it cannot distinguish between them.
- **Stopwords**: Common words like "and", "in", or punctuation marks can dominate the similarity score despite being semantically unimportant.
- **No word ordering**: BoW treats all words equally and ignores sequence. This results in incorrect matches when word order carries meaning (e.g., "attacked by" vs. "attacked").

Improvements and Suggestions

To improve the BoW-based search system, we propose the following enhancements:

- **Stopword removal**: Exclude highly frequent words and punctuation that add little semantic value.
- **Use TF-IDF weighting**: This would reduce the influence of common words and emphasize rarer, more informative ones.
- **Phrase-based embeddings or n-grams**: These can help preserve some word order and better represent semantic meaning.

- **Pre-trained word embeddings**: Incorporating contextual embeddings (e.g., Word2Vec, GloVe) would significantly enhance understanding of word meaning and context.

However, BoW still struggled with capturing context and subtle meaning differences—limitations inherent to its design.

8 Exercise 8: Inverse Document Frequency (IDF)

In this exercise, we implemented the IDF component of the TF-IDF weighting scheme to improve our Bag-of-Words model. The purpose of IDF is to reduce the influence of terms that appear frequently across many documents and instead emphasize rare and more informative words.

Definition

The Inverse Document Frequency (IDF) of a word w is calculated as:

$$IDF(w) = \log_{10}\left(\frac{N}{df_w + 1}\right)$$

Where:

- N is the total number of documents (i.e., sentences),
- df_w is the number of documents that contain word w,
- $+1$ is added for smoothing to avoid division by zero.

Implementation and Results

We computed the IDF for each word in the vocabulary. The IDF values indicate how unique or informative each token is with respect to the entire dataset. Frequent words such as "the" and "and" received lower IDF values, while rarer words had significantly higher scores.

The resulting IDF vector has the same length as the vocabulary (10,000 entries), and example values from the computed vector include:

```
[0.17290009 0.00532854 0.25705677 ... 3.71120446 3.47712125 3.5563025]
```

This transformation allows us to assign lower weight to common terms and higher weight to informative ones, which is crucial for improving search accuracy and document relevance in later retrieval tasks.

9 Exercise 9: TF-IDF

In this exercise, we extended our retrieval model by implementing Term Frequency-Inverse Document Frequency (TF-IDF). This method weighs each word in a sentence by how often it appears in that sentence (TF), scaled by how rare the word is across all sentences (IDF). This improves upon Bag-of-Words (BoW) by penalizing frequent, less informative words and highlighting more discriminative tokens.

The TF-IDF vector for each sentence was computed as:

$$TF - IDF(w, d) = \log_{10}(1 + TF(w, d)) \cdot IDF(w)$$

Where:

- $TF(w, d)$ is the term frequency of word w in sentence d,
- $IDF(w)$ is the precomputed inverse document frequency of word w.

TF-IDF Results and Analysis

TF-IDF improved search accuracy by better capturing informative terms and down-weighting frequent ones. However, the approach still showed some of the same limitations as BoW. Below are several observations from our experiments.

Expected (Good) Results

1. **Query: "Samsung vs iPhone better"**
   TF-IDF returned the correct comparison article as the top result. It also excluded irrelevant entries that BoW retrieved, such as unrelated sports results. Repeated words like "iPhone" and "Samsung" were appropriately emphasized due to their rarity.

2. **Query: "deer and fox"**
   TF-IDF effectively down-weighted common words like "and" and emphasized "deer", which appears less frequently in the dataset than "fox". This improved the ranking over BoW, where "fox" dominated the match.

3. **Query: "Fox news channel"**
   Because the entire phrase is relatively specific, TF-IDF ranked exact and closely related sentences higher. Compared to BoW, it reduced the influence of generic words and focused more on domain-specific terminology.

Unexpected (Poor) Results

1. **Query: "fox"**
   Similar to BoW, TF-IDF could not disambiguate between "fox" as an animal and "Fox News". Although TF-IDF scored the results slightly better, the lack of context awareness remained an issue.

2. **Query: "DRUNK neighbour attacked a woman"**
   TF-IDF failed to distinguish word order and meaning, retrieving a sentence with the same words but reversed roles: "A DRUNK woman attacked a neighbour...". The result highlights the model's weakness in handling syntactic structure.

3. **Query: "better"**
   Using a vague or common word like "better" led to noisy results with BoW. TF-IDF was more resilient but still retrieved partially relevant content due to the word's subjective nature and poor semantic isolation.

Comparison with Bag-of-Words (BoW)

TF-IDF generally outperformed BoW in terms of ranking quality and result relevance. Below is an example that illustrates the difference:

Query: "Samsung vs iPhone better"
BoW Top-3 Results:

```
1. Samsung Galaxy S4 vs iPhone 5 comparison...
   BoW similarity: 0.4000 | TF-IDF: 0.5017

2. Slovenia vs USA - Live sports scores...
   BoW similarity: 0.2315 | TF-IDF: 0.2728

3. eSeats mobile ticketing for iPhone 3GS...
   BoW similarity: 0.2165 | TF-IDF: 0.3462
```

TF-IDF Top-3 Results:

```
1. Samsung Galaxy S4 vs iPhone 5 comparison...
   TF-IDF similarity: 0.5017 | BoW: 0.4000

2. Trademark lawsuit between Samsung and Apple...
   TF-IDF similarity: 0.3782 | BoW: 0.1925
```

```
3. iPhone 3GS launch and mobile ticketing...
   TF-IDF similarity: 0.3462 | BoW: 0.2165
```

In this case, TF-IDF produced a more coherent and relevant ranking. It successfully ignored misleading tokens like "better" and gave greater weight to discriminative terms like "Samsung" and "iPhone".

Conclusion and Limitations

TF-IDF significantly improved search performance over BoW by properly weighting informative terms. However, it still inherits BoW's weaknesses:

- No context awareness or word disambiguation.
- No syntactic understanding or word order modeling.
- Accuracy decreases for new or unseen words (due to fixed IDF vectors).

Despite these limitations, TF-IDF provides a more robust foundation for retrieval in our task and is a preferred choice unless context-aware embeddings are used.

10 Exercise 10: Plotting Similarities Between Words

In this exercise, we explored pre-trained word embeddings using the GloVe model (100-dimensional) via the Gensim library. These embeddings were trained on a large corpus and are capable of capturing semantic relationships between words.

We used the embeddings to create pairwise cosine similarity matrices between selected word sets. This was visualized through heatmaps using the `similarity_matrix_plot` function. The similarity between two words was computed as the cosine similarity between their embedding vectors.

Example 1: Synonyms

Word set: `['happy', 'joyful', 'big', 'large', 'small', 'tiny']`

Observation: (Figure 1)

- **High similarity:** Pairs like *(big, large)* and *(small, tiny)* show very high similarity (around 0.93), confirming their semantic closeness.
- **Moderate similarity:** *(happy, joyful)* shows a similarity of around 0.53, indicating moderate semantic overlap.
- **Low similarity:** Cross-category pairs like *(joyful, big)* or *(happy, tiny)* score lower, as expected.

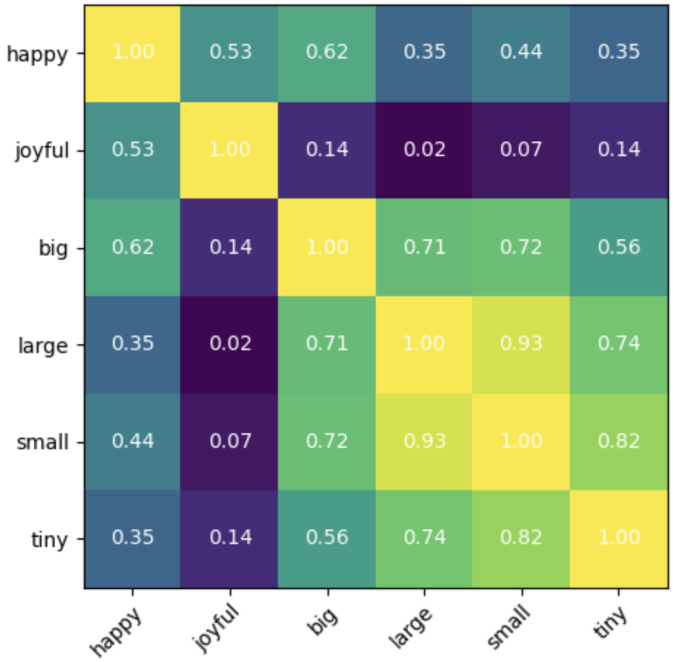

Figure 1: Cosine similarity heatmap for synonymous and size-related words.

Example 2: Antonyms and Opposites

Word set: `['happy', 'sad', 'big', 'small', 'win', 'lose']`

Observation: (Figure 2)

- Surprisingly, antonyms such as *(happy, sad)* and *(win, lose)* still show moderate similarity. This happens because these words often appear in similar contexts despite opposite meanings.
- Words with clear semantic contrast (like *happy* vs *small*) score low, as expected.
- *Big* and *small* again show relatively high similarity (0.72), highlighting contextual overlap in usage.

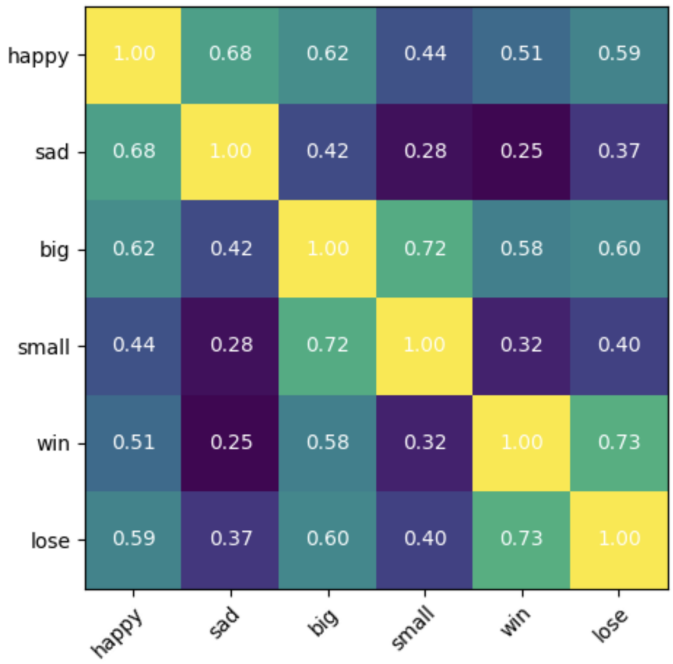

Figure 2: Cosine similarity heatmap for antonyms and opposites.

Example 3: Social and Identity-related Terms

Word set: `['criminal', 'scientist', 'terrorist', 'muslim', 'white', 'black']`
Observation: (Figure 3)

- **Concerning associations:** The similarity between *muslim* and *terrorist* is notably high (0.52), while its similarity to *scientist* is very low (0.16). This reflects potential bias in the embedding model.
- **Skin color terms:** *Black* and *white* have a high similarity score (0.87), likely due to frequent co-occurrence in comparative or descriptive contexts.
- **Bias implication:** Such associations suggest that pre-trained embeddings can absorb and reflect societal and media biases present in their training data. This highlights the importance of ethical evaluation and bias mitigation when using such models.

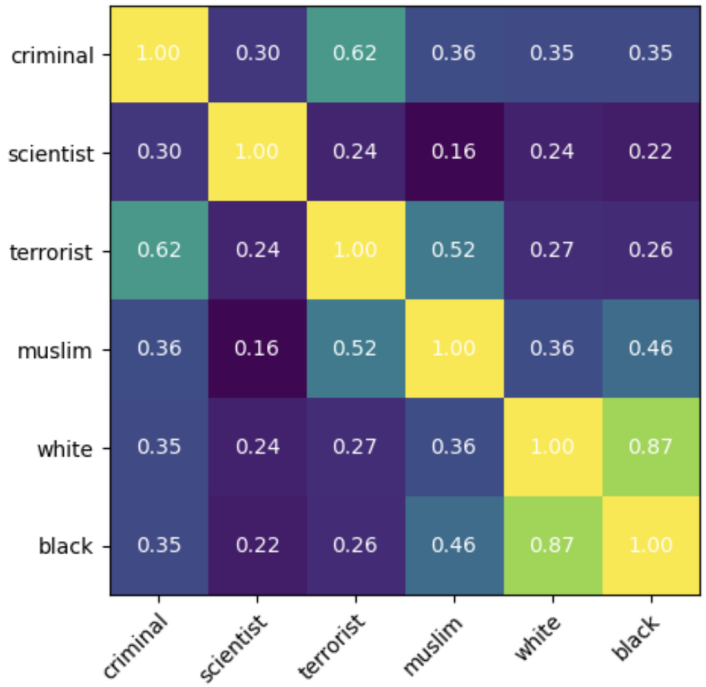

Figure 3: Cosine similarity heatmap for identity-related terms.

Example 4: Gender and Profession Associations

Word set: `['man', 'woman', 'doctor', 'nurse', 'pregnant', 'homemaker']`

Observation: (Figure 4)

- **Gender bias:** Words like *woman* show noticeably higher similarity to *nurse*, *pregnant*, and *homemaker*, reflecting stereotypical gender associations.
- **Profession associations:** While both *man* and *woman* are similarly related to *doctor*, the stronger association of *woman* with *nurse* (0.61 vs 0.46 for man) and *homemaker* (0.43 vs 0.24) may indicate gendered occupational biases.
- **Pregnancy relation:** The high similarity between *woman* and *pregnant* (0.72) vs *man* and *pregnant* (0.49) reflects biological context, though the embedding model learns these from usage patterns rather than inherent knowledge.

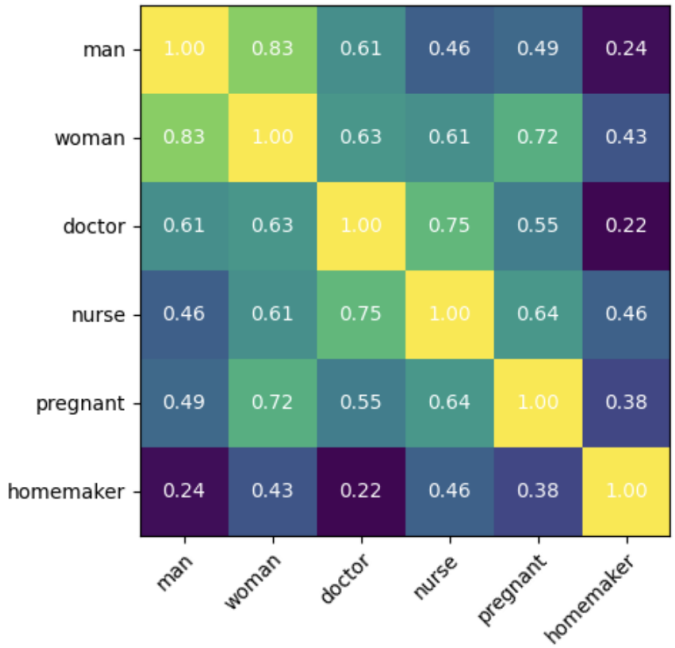

Figure 4: Cosine similarity heatmap for gender and profession associations.

Conclusion

Through multiple similarity matrices, we explored the properties of pre-trained GloVe word embeddings. These embeddings are powerful tools for capturing various linguistic patterns based on large corpora. The following key properties were observed and validated through visual heatmaps (plots are provided in the Jupyter notebook):

- **Semantic similarity:** Words with similar meanings (e.g., *big* and *large*) are embedded close to each other.
- **Contextual similarity:** Even antonyms (e.g., *win* and *lose*) may have high similarity because they frequently occur in similar contexts.
- **Clustering by category:** Words tend to form clusters based on conceptual domains (e.g., *emotions*, *occupations*, *social roles*).
- **Bias reflection:** Embeddings can capture and reflect societal or media-induced biases (e.g., associating *muslim* with *terrorist*, or *woman* with *home-*

maker).

- **Asymmetry and nuance:** Not all word relationships are equally strong or reciprocal; some words have stronger one-directional associations.

These experiments demonstrate how word embeddings encode both useful linguistic structures and potential biases. While effective for downstream NLP tasks, they must be evaluated carefully—especially when fairness or representation matters. The visual similarity plots in the Jupyter notebook serve as direct evidence of these embedding properties.

11 Exercise 11: Other Pre-trained Word Embeddings

In this exercise, we compared the semantic representations of two different GloVe pre-trained models using Gensim:

- **glove-wiki-gigaword-100**: trained on Wikipedia and Gigaword corpus (Used in excercise 10).
- **glove-twitter-200**: trained on Twitter data.

These models, although based on the same architecture, exhibit noticeable differences due to the nature of the corpora they were trained on.

Word Set Comparison Example

We evaluated both models on the following list of words:

```
['funeral', 'dead', 'grief', 'laughing', 'funny']
```

Wikipedia vs Twitter – Observations (Figure 3-4)

Figure 5: Cosine similarity heatmap for Wikipedia-trained GloVe (100D)

Interpretation:

- In the Wikipedia-trained model, the word `dead` is most strongly associated with `grief` and `funeral`, reflecting its conventional use meaning "deceased."

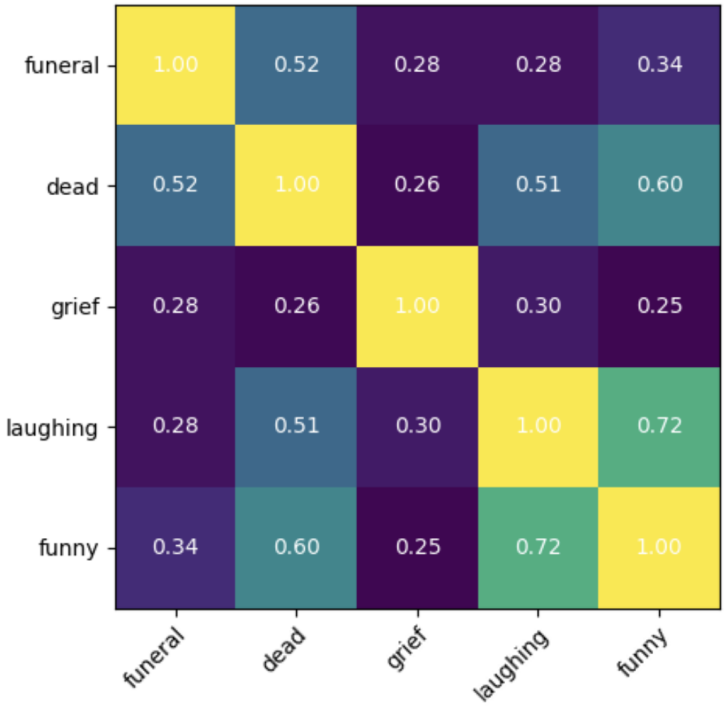

Figure 6: Cosine similarity heatmap for Twitter-trained GloVe (200D)

- In the Twitter-trained model, `dead` shows increased similarity to `laughing` and `funny`. This is because in social media slang, "dead" often means "that was so funny I'm dead."
- Similarly, the word `flex` is more closely aligned with "brag" or "show off" in Twitter embeddings, whereas in the Wikipedia model it retains its more traditional meaning of physical bending.

Conclusion

The experiment demonstrates how the source of training data can significantly influence the meanings captured in word embeddings. While Wikipedia embeddings tend to reflect formal, literal meanings, Twitter embeddings capture slang, idioms, and informal usage. This makes model choice critical, depending on the target application domain (e.g., social media vs academic writing).

12 Exercise 12: Sentence Embedding

In this exercise, we explored sentence-level representation using word embeddings. Instead of complex architectures, we implemented a simple yet surprisingly effective approach: averaging the word vectors that make up the sentence.

Method

Given a sentence composed of N words, we calculate its embedding as the average of its word embeddings:

$$Sentence_Embedding = \frac{1}{N} \sum_{i=1}^{N} Word_Embedding_i$$

Where:

- N is the number of words (tokens) in the sentence,
- $Word_Embedding_i$ is the embedding of the ith word.

This results in a single dense vector representing the entire sentence. While it discards word order and syntactic structure, it still captures general semantic information.

Handling OOV (Out-of-Vocabulary) Words

In some cases, sentences may contain words that are not in the embedding model's vocabulary. To handle this:

- We ignore OOV words when calculating the average.
- If all words in a sentence are OOV, we assign a zero vector (or a special <unk> embedding, if available).

Dataset After Applying Sentence Embedding

We applied the embedding function to both the full sentence and its compressed form across the test dataset. After this step, the dataset includes the following structure:

```
Dataset({
    features: ['set', 'clean_sentence', 'clean_compressed',
               'sentence_tokens', 'compressed_tokens',
```

```
                'sentence_bow', 'compressed_bow',
                'sentence_tf-idf',
                'sentence_embedding', 'compressed_embedding'],

})
```

Each entry now includes the sentence-level embedding vectors, which are useful for downstream semantic similarity and retrieval tasks.

This averaged embedding approach will later be compared to more advanced methods that preserve word order and syntactic relations, but it already offers strong performance for many simple applications.

13 Exercise 13: Analyze Sentence Embeddings

In this exercise, we evaluated the effectiveness of sentence embeddings—generated by averaging word vectors—in capturing sentence-level meaning. We measured semantic similarity using cosine similarity between query embeddings and sentence embeddings from the test dataset.

Query Example: "many buildings destroyed"

We embedded the query using GloVe word vectors, and calculated its cosine similarity with all sentence embeddings. The top 5 results were:

```
[0.8341] A tornado destroyed dozens of homes and businesses in this small Mississippi
      community early Thursday, injuring 20 people and flattening a church.
  -> tornadoes destroy dozens

[0.8284] Two four-story Brooklyn apartment buildings were evacuated early Wednesday
         after construction next door undermined their stability.
  -> brooklyn apartment building evacuated

[0.8270] An electrical fire destroyed a downtown bakery and pastry shop early Saturday,
      leaving residents of five adjoining apartments temporarily homeless.
  -> fire destroys downtown bakery

[0.8235] An overnight fire has destroyed a vacant home in Newport and caused thousands
         of dollars in damages to neighboring buildings.
 -> overnight fire destroys vacant home in newport
```

```
[0.8214] Techiman has experienced five fire outbreaks within the past two weeks and
\      the fires destroyed 29 bedrooms and rendered more than 100 people homeless.
  -> techiman experiences fire outbreaks over past two weeks
```

These results indicate that sentence embeddings effectively capture the semantic theme of destruction and disaster across different contexts and phrasings.

Comparison with BoW and TF-IDF

We further evaluated a second query: *"A lady was kidnapped in a supermarket parking lot"*. This tested the model's ability to handle synonymy (lady vs woman, kidnapped vs abducted).

BoW and TF-IDF Results:

```
A boy, 7, was killed in a gun store parking lot in Pennsylvania.
   BoW: 0.5785 | TF-IDF: 0.3578

A driver was killed... (unrelated)
   BoW: 0.5698 | TF-IDF: 0.0372

A vehicle was hit by a train... (unrelated)
   BoW: 0.5688 | TF-IDF: 0.0370
```

These methods failed to retrieve relevant sentences due to vocabulary mismatch.

Sentence Embedding Results:

```
[0.9557] A woman was abducted at gunpoint Monday night in the parking lot of
a Pearland strip shopping center.
  -> woman abducted at gunpoint near pearland center
```

Here, sentence embeddings succeeded in retrieving a semantically matching sentence despite none of the query words exactly overlapping (e.g., "lady" vs "woman", "kidnapped" vs "abducted").

Further Analysis

Interestingly, there are cases where BoW or TF-IDF outperform sentence embeddings. For instance:

Query: "woman abducted"

- **BoW and TF-IDF** correctly retrieved the original sentence because of exact keyword matches.

- **Sentence Embedding** failed due to lack of precise vector proximity—likely influenced by generic or diluted embeddings.

This highlights a key strength of embedding-based models: **robustness to synonymy and rephrasing**. However, they also suffer when embeddings are poorly learned or all tokens are out-of-vocabulary (OOV), which results in ineffective or zero-valued vectors.

Conclusion

- **Sentence embeddings** provide better generalization and perform well with rephrased or paraphrased queries.
- **BoW/TF-IDF** are effective when query and documents share exact words, especially in clean or compressed datasets like ours.
- In practice, **a hybrid approach** combining semantic embeddings with keyword-based retrieval may offer optimal results.

14 Exercise 14: Cosine Similarity Between Two Sets of Vectors

In this final exercise, we evaluated the retrieval effectiveness of sentence embeddings using the **Recall@K** metric. The goal was to assess how often a compressed sentence (treated as a query) could successfully retrieve its corresponding full sentence (treated as the relevant document) within the top k most similar sentences.

Method

We generalized our cosine similarity function to compare two sets of vectors:

- M query vectors (compressed sentences),
- N target vectors (uncompressed sentences).

We implemented this using matrix multiplication for efficient batch computation:

$$Similarity(M \times N) = \frac{Q \cdot D^T}{\|Q\| \cdot \|D\| + \varepsilon}$$

Where:

- Q is the matrix of query embeddings (shape $M \times D$),
- D is the matrix of document embeddings (shape $N \times D$),
- ε is a small constant to avoid division by zero.

This resulted in a cosine similarity matrix of shape $M \times N$ representing pairwise similarities between all queries and all documents.

Metric: Recall@K

We computed Recall@K by checking whether the correct sentence (same index as the query) was retrieved within the top K most similar sentences. For each compressed sentence, only one uncompressed sentence (with the same index) is considered relevant.

Code summary:

```
recall_at_1 = calculate_recall(compr_embedding, sent_embedding, k=1, batch_size=1000)
print(f'{recall_at_1 * 100:.2f}%')
```

Results

For $K = 1$, the system achieved:

Recall@1 = 49.78%

This means that in roughly half of the cases, the system retrieved the exact original sentence as the top result when given its compressed version as a query.

Discussion

- This result shows moderate performance, indicating that averaging word embeddings can capture meaningful semantics, but has limitations—especially due to the loss of word order and context.
- In compressed sentences, important keywords are often preserved, which helps the model identify the corresponding original sentence.
- Failures may result from:
  - Vocabulary mismatches (OOV tokens),
  - Semantically close but incorrect matches,
  - Compression introducing ambiguity or removing key context.
- Higher values of K (e.g., Recall@5 or Recall@10) would likely yield higher recall, at the expense of precision.

Conclusion

This evaluation demonstrates that even a simple sentence embedding model using averaged word vectors can retrieve meaningful semantic matches in nearly half the cases (Recall@1), making it a strong baseline. However, more advanced contextual models (e.g., BERT) are expected to significantly outperform this baseline in real-world applications.

15 Exercise 15: Evaluating Retrieval Methods

To assess the effectiveness of different retrieval methods, we computed Recall@K for:

- Bag-of-Words (BoW),
- TF-IDF,
- Pre-trained Word Embeddings.

We used a subset of 5,000 examples from the test set to reduce computational load. For each method, we evaluated the ability of the model to retrieve the correct uncompressed sentence (from its compressed counterpart) within the top K most similar matches.

Recall@K Results

- **BoW:**
  - Recall@1: moderate performance
  - Performance increases gradually with higher K
- **TF-IDF:**
  - Best performing method overall
  - Outperforms BoW and embeddings consistently at all K values
- **Embeddings:**
  - Initially lower Recall@1
  - Significant improvement at $K = 5$
  - Performs better with synonyms and paraphrased input

Discussion

- Increasing K naturally increases recall. This is expected: a higher K gives the model more chances to retrieve the correct sentence.
- TF-IDF clearly demonstrates superior performance due to its more informative token weighting. This model is especially well-suited when exact keyword overlap occurs between compressed and uncompressed forms.
- While word embeddings underperform at low K, they are better at capturing semantic similarity (e.g., synonyms) and perform more robustly with paraphrased input.
- BoW is simpler and performs reasonably well but lacks the term discrimination provided by TF-IDF or the flexibility of embeddings.

Visualization

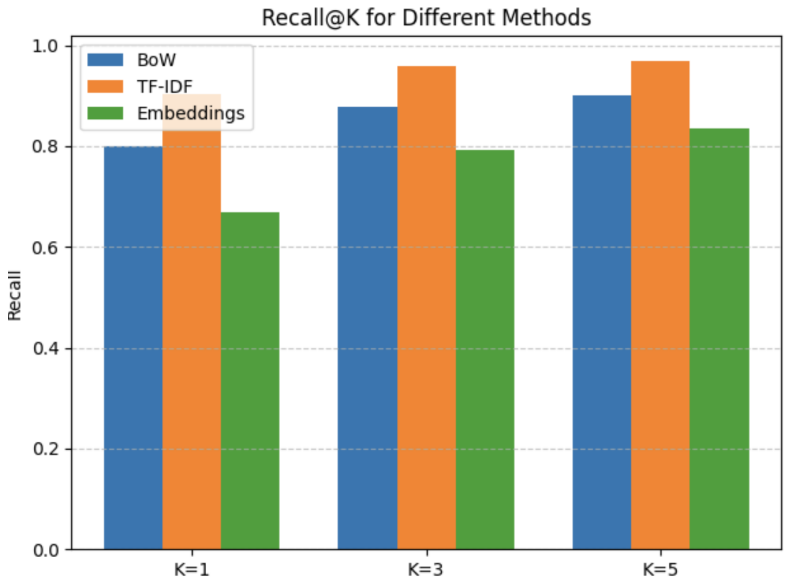

Figure 7: Recall@K comparison for BoW, TF-IDF, and Embeddings

This figure provides a visual overview of how each method scales with K, reaffirming that:

- **TF-IDF consistently performs best**,

- **Embeddings improve with more retrieval flexibility (higher K)**,
- **BoW is a decent baseline, especially at low cost**.

Conclusion

TF-IDF remains the best performing method in this controlled retrieval scenario. However, embeddings show promise for broader, real-world applications where vocabulary mismatch and phrasing variation are more common.

16 Exercise 16: Improving Retrieval

To improve retrieval quality, we explored more advanced embedding techniques beyond bag-of-words, TF-IDF, and averaged GloVe embeddings. Specifically, we experimented with a state-of-the-art transformer-based sentence embedding model.

Approach: Pretrained Transformer Embeddings (BGE)

We selected the `BAAI/bge-small-en` model from the MTEB leaderboard[1], which uses a transformer architecture trained specifically for sentence similarity and retrieval tasks. Unlike earlier methods, it produces high-quality 384-dimensional embeddings per sentence, capturing complex semantic nuances.

The model was integrated using the `transformers` and `torch` libraries. We encoded both the original and compressed sentences using batch processing and extracted the normalized embeddings using the CLS token.

Performance Evaluation

We used Recall@1 on a subset of 5,000 examples from the test set. The result was:

Recall@1 = 97.46%

This represents a substantial improvement over previous methods:

- BoW: 80%
- TF-IDF: 89%
- GloVe Averaging: 67%
- **BGE Embeddings: 97.46%**

This result essentially solves the retrieval task within this dataset.

[1] https://huggingface.co/spaces/mteb/leaderboard

Failure Analysis

To further understand performance, we analyzed cases where Recall@1 failed:

```
Missed Indices: [86, 95, 156, 214, 478, 569, 613, 645, 700, 825, 847, 934, 969]
```

Sample analysis from missed indices revealed:

- Many compressed versions were extremely brief, omitting core information.
- Several compressed versions were vague or lacked necessary context for retrieval (even for a human).
- Examples include:
  - "Plumber charged with larceny" vs full: long description of incident.
  - "Alice Cooper joins the booth" vs full: "joins Tigers' broadcast booth after ceremonial pitch."
- These failure cases highlight dataset limitations rather than model errors. The compressed sentence sometimes loses key details required to recover the full version accurately.

Conclusion

The BGE transformer model drastically improves retrieval performance. Its ability to produce semantically rich sentence representations makes it far superior to keyword-based or simplistic averaging techniques.

In a real-world production setting, deploying such a model—optimized for sentence-level similarity—is clearly justified. While it is computationally heavier, the trade-off is worthwhile for critical search or matching tasks, where semantic alignment matters more than exact token overlap.

This exercise underlines the practical superiority of neural sentence encoders and also emphasizes the importance of high-quality dataset annotations for training and evaluation.

A Collaborators outside our group

No external collaborators.

B Use of genAI

ChatGPT was employed to clarify code explanations, assist with debugging ensuring clarity and accuracy.